

Retrieval-Augmented Generation Fundamentals

Chapter 1: What Is RAG?

Retrieval-Augmented Generation, or RAG, combines information retrieval with a language model. Instead of asking the language model to answer only from its internal knowledge, the system first retrieves relevant information from an external document collection.

A basic RAG pipeline contains document loading, text chunking, embedding generation, vector storage, retrieval, prompt construction, and answer generation.

Key Points

- RAG connects retrieval with generation.
- External documents provide grounding context.
- The retriever and generator have different responsibilities.

Chapter 2: Retrieval and Context

Documents are divided into chunks because a large document is difficult to search and pass entirely to a language model. Each chunk can be represented by an embedding, which is a numerical vector capturing semantic information.

A vector database or vector index stores embeddings and supports similarity search. When a user asks a question, the question is embedded using the same embedding model and compared with document vectors.

The top-k results are selected as context. Metadata such as source filename, page number, and chunk identifier can travel with each chunk so that the application can display sources and debug retrieval quality.

Key Points

- Chunk size affects retrieval quality.
- Top-k controls how many retrieved chunks are passed to generation.
- Metadata helps trace an answer back to source documents.

Chapter 3: Grounded Answer Generation

The generator receives a prompt containing the user question and retrieved context. A strong RAG prompt tells the model to use the supplied context and avoid inventing information that is not supported by the retrieved documents.

A production RAG system should measure retrieval quality separately from generation quality. If the correct information is never retrieved, improving the generation prompt cannot fully solve the problem.

Source citations, logging, evaluation datasets, retry handling, and access controls are useful additions when moving from a demo to a production system.

Key Points

- Ground the answer in retrieved context.
- Evaluate retrieval and generation separately.
- Return source information when possible.